

Homework #5

April 29, 2021

Chun-Yuan (Scott) Chiu
chunyuac@andrew.cmu.edu

1.

(a)

$$\begin{aligned} E(Y_t | X_t = 0.1, a_{t-1} = 0.6) &= E(\beta_0 + \beta_1 X_t + \delta \sigma_t^2 + a_t \mid X_t = 0.1, a_{t-1} = 0.6) \\ &= E\left(\beta_0 + \beta_1 X_t + \delta(1 + 0.4a_{t-1}^2) + \epsilon_t \sqrt{1 + 0.4a_{t-1}^2} \mid X_t = 0.1, a_{t-1} = 0.6\right) \\ &= E\left(0.06 + 0.3 \times 0.1 + 0.2 \times (1 + 0.4 \times 0.6^2) + \epsilon_t \sqrt{1 + 0.4 \times 0.6^2}\right) \\ &= 0.3188. \end{aligned}$$

(b)

Following part (a), we have

$$\begin{aligned} Var(Y_t | X_t = 0.1, a_{t-1} = 0.6) &= Var\left(0.06 + 0.3 \times 0.1 + 0.2 \times (1 + 0.4 \times 0.6^2) + \epsilon_t \sqrt{1 + 0.4 \times 0.6^2}\right) \\ &= (1 + 0.4 \times 0.6^2) Var(\epsilon_t) \\ &= (1 + 0.4 \times 0.6^2) \\ &= 1.144. \end{aligned}$$

(c)

Yes, $Y_t | X_t, a_{t-1}$ is normal. Since

$$Y_t = \beta_0 + \beta_1 X_t + \delta(1 + 0.4a_{t-1}^2) + \epsilon_t \sqrt{1 + 0.4a_{t-1}^2},$$

given the values of X_t, a_{t-1} , and the parameters β_0, β_1 and δ , Y_t is simply a shifted scaled ϵ_t which is normal.

(d)

No, in general the unconditioned marginal distribution of Y_t is not normal. As we have seen in the lecture, as ϵ_t has a standard normal distribution and $\alpha < 0.577$ (which is satisfied here since $\alpha = 0.4$), a_t already has a heavier tail than normal. Thus a_t is not normal and neither is Y_t .

2.

(a)

We download the data, compute the log-return and run the augmented Dickey-Fuller test. The result indicates there is no unit root in the time series.

```
[1]: import statsmodels.tsa.api as smt
import pandas_datareader as pdr
import numpy as np

ticker = 'mrk'
start = '2011-01-01'
end = '2021-04-23'
shf = 1

adjclose = pdr.get_data_yahoo(ticker, start, end)['Adj Close']
logret = np.log(adjclose/adjclose.shift(shf)).dropna()

smt.stattools.adfuller(logret)
```

```
[1]: (-13.533680410315691,
      2.592290384924904e-25,
      15,
      2577,
      {'1%': -3.4328900953577164,
       '5%': -2.8626622153764103,
       '10%': -2.5673673962076693},
      -15057.762642995938)
```

(b)

We then present the time series plots, and the ACF and the PACF plots. We do this 3 times for data from different windows: the full period 1/1/2011 - 4/23/2021, the first half of the full period, and the second half. As a result, the ACF of the full period data is very similar to the one only given the second half, but not so much with the first half. Since the unusual 2020 March market condition is only in the second half but not the first half, it makes sense for them to have different autocorrelation structures.

```
[2]: from pandas import Series
import matplotlib.pyplot as plt
import statsmodels.tsa.api as smt
import warnings

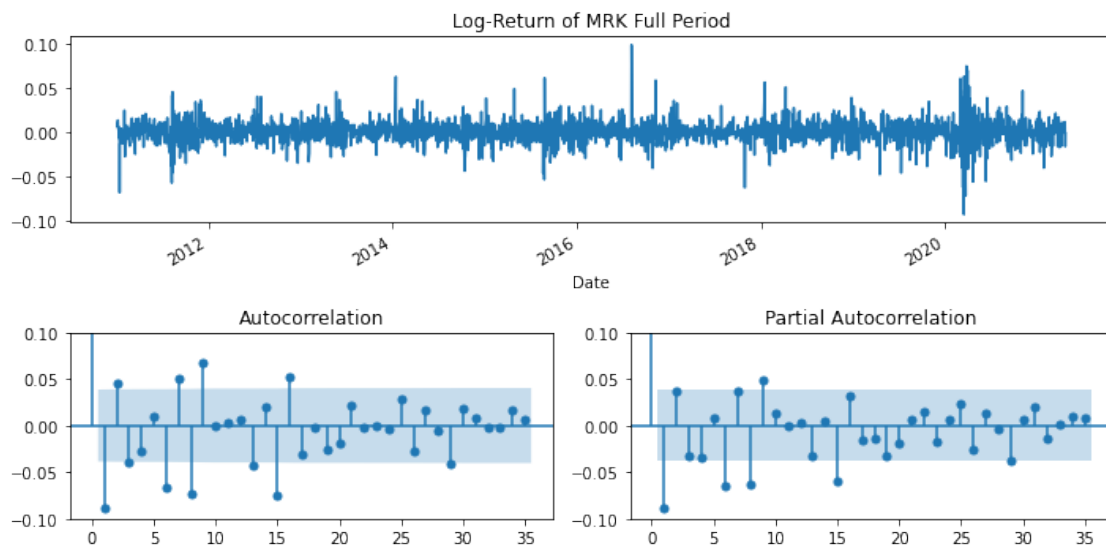
warnings.filterwarnings('ignore')

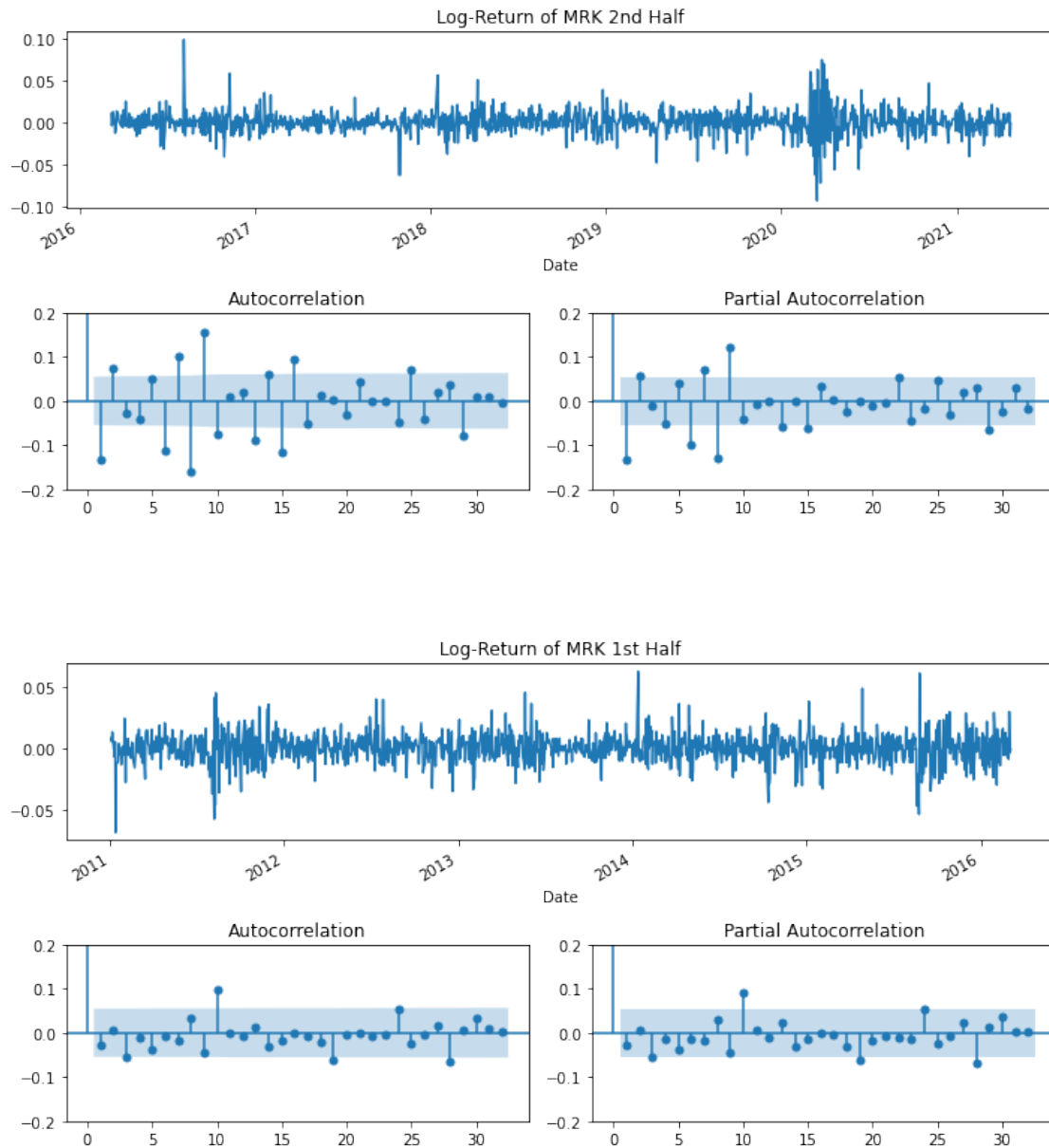
def tsplot(ts, title, acf_ylim=(-0.2, 0.2)):
    fig = plt.figure(figsize=(10, 5))
    layout = (2,2)
    ts_ax = plt.subplot2grid(layout, (0,0), colspan=2)
    pacf_ax = plt.subplot2grid(layout, (1,1))
    acf_ax = plt.subplot2grid(layout, (1,0))

    Series(ts).plot(ax=ts_ax)
    smt.graphics.plot_acf(ts, ax=acf_ax)
    smt.graphics.plot_pacf(ts, ax=pacf_ax)
    ts_ax.set(title=title)
    pacf_ax.set(ylim=acf_ylim)
    acf_ax.set(ylim=acf_ylim)

    plt.tight_layout()
    plt.show()

tsplot(logret, title='Log-Return of MRK Full Period', acf_ylim=(-0.1, 0.1))
tsplot(logret[1300:], title='Log-Return of MRK 2nd Half', acf_ylim=(-0.2, 0.2))
tsplot(logret[:1300], title='Log-Return of MRK 1st Half', acf_ylim=(-0.2, 0.2))
```





The Ljung-Box test results suggest that there is nonzero autocorrelation starting lag 1.

```
[3]: from statsmodels.stats.diagnostic import acorr_ljungbox
acorr_ljungbox(logret, lags=10, return_df=True)
```

```
[3]:      lb_stat      lb_pvalue
1    19.962041  7.899491e-06
2    25.152498  3.453062e-06
3    29.245462  1.988620e-06
```

4	31.129532	2.880669e-06
5	31.397424	7.818030e-06
6	43.100199	1.114373e-07
7	49.743647	1.621907e-08
8	63.672652	8.826790e-11
9	75.247452	1.412127e-12
10	75.248326	4.256608e-12

(c)

The model selection shows that ARMA(4, 2) minimizes both AIC and BIC.

```
[13]: import statsmodels.tsa.api as smt

res = smt.stattools.arma_order_select_ic(logret, ic=['aic', 'bic'], trend='nc')
print('AIC min order: ', res.aic_min_order)
print('BIC min order: ', res.bic_min_order)
```

AIC min order: (4, 2)

BIC min order: (4, 2)

The fitted model parameters are in this order: Constant, ARMA parameters, and σ_t^2 .

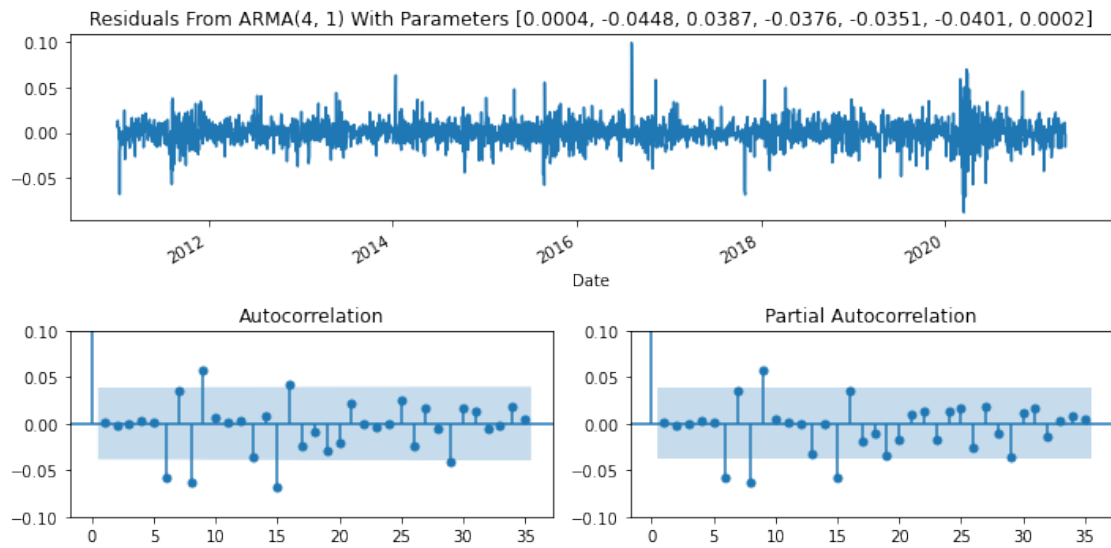
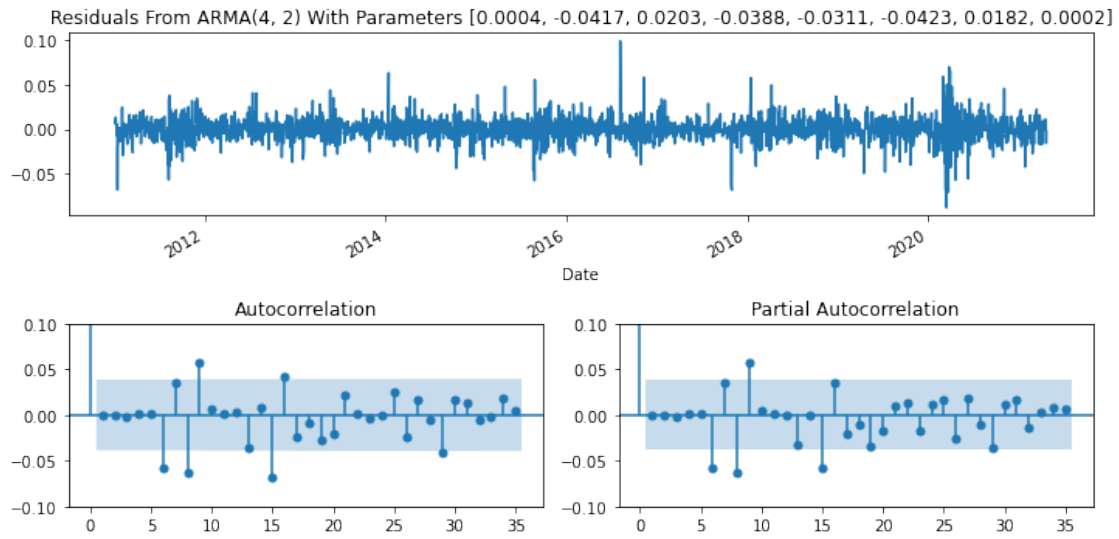
```
[4]: from statsmodels.tsa.arima.model import ARIMA

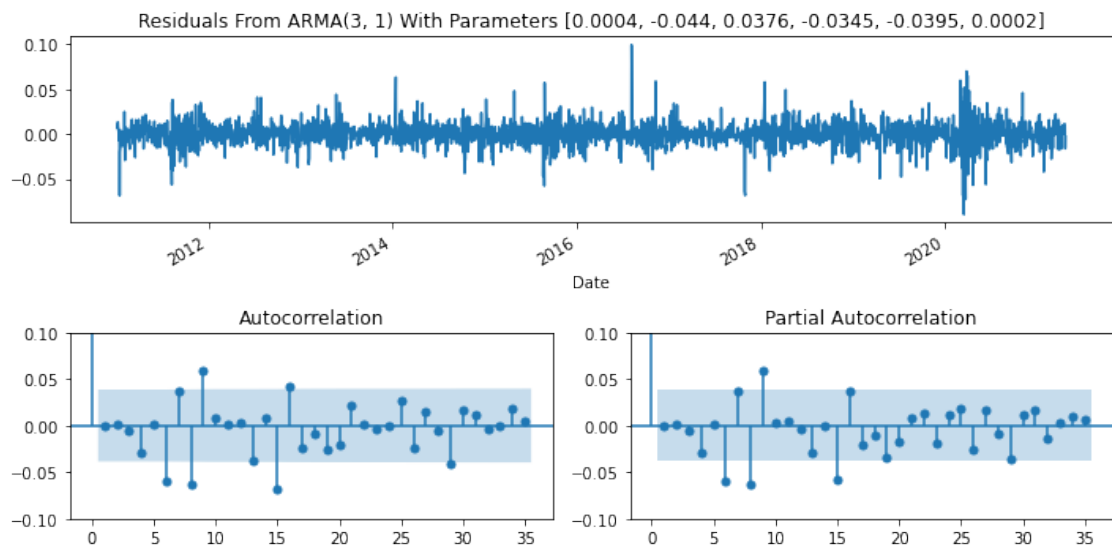
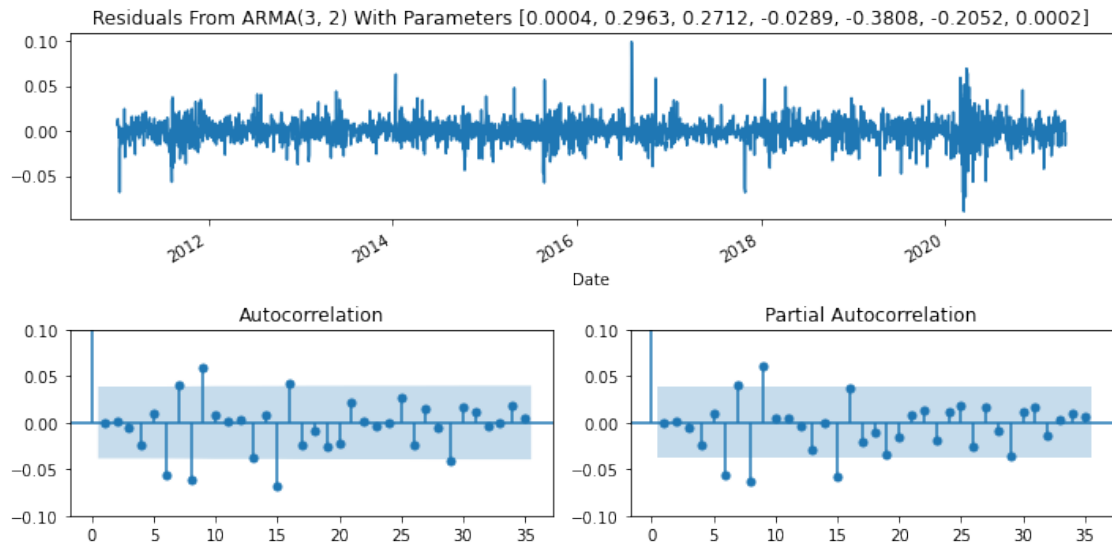
ARIMA(logret, order=(4, 0, 2)).fit().params
```

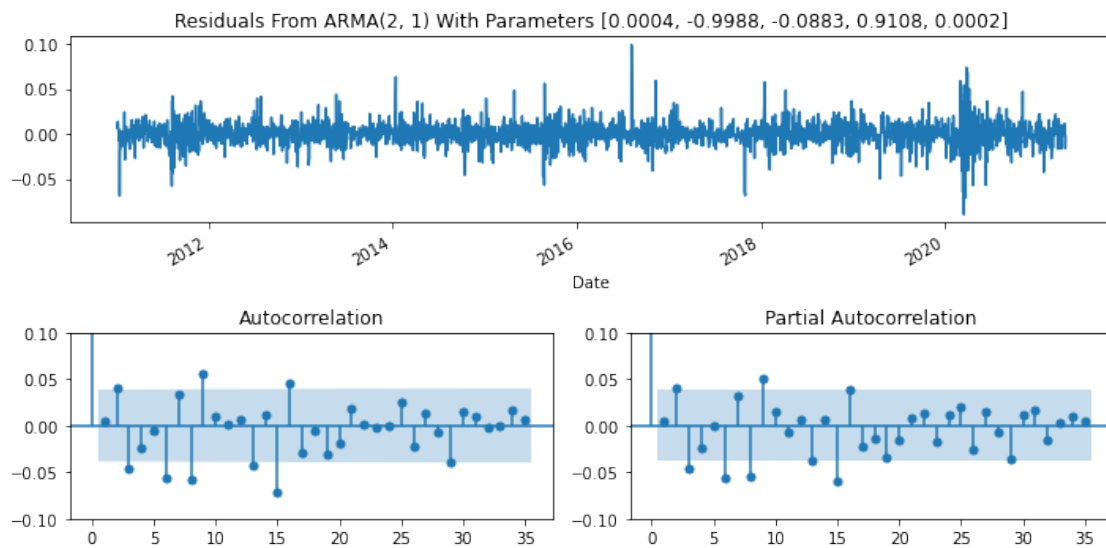
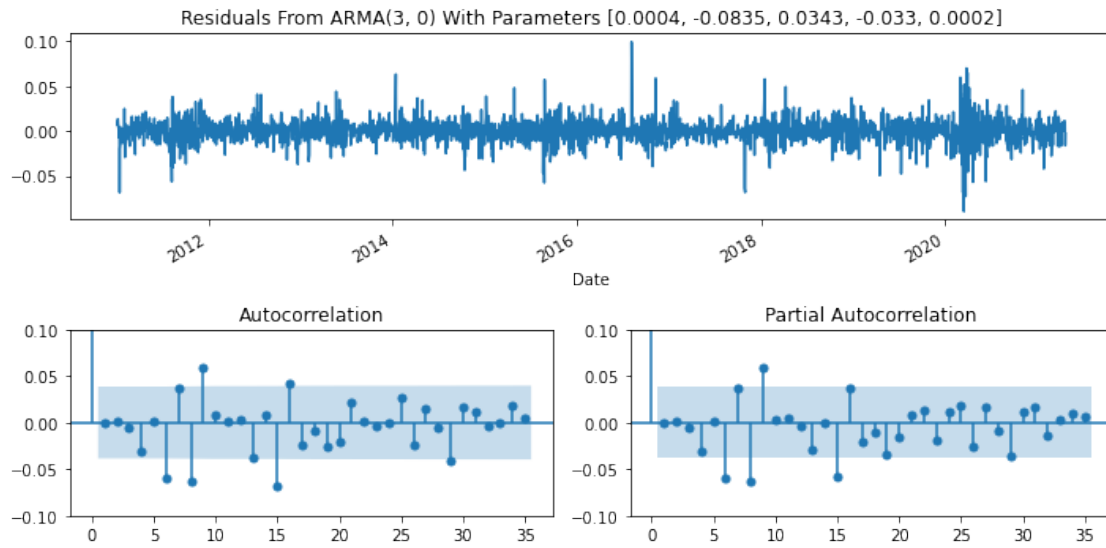
```
[4]: const      0.000438
ar.L1      -0.041699
ar.L2       0.020261
ar.L3      -0.038786
ar.L4      -0.031051
ma.L1      -0.042317
ma.L2       0.018234
sigma2     0.000167
dtype: float64
```

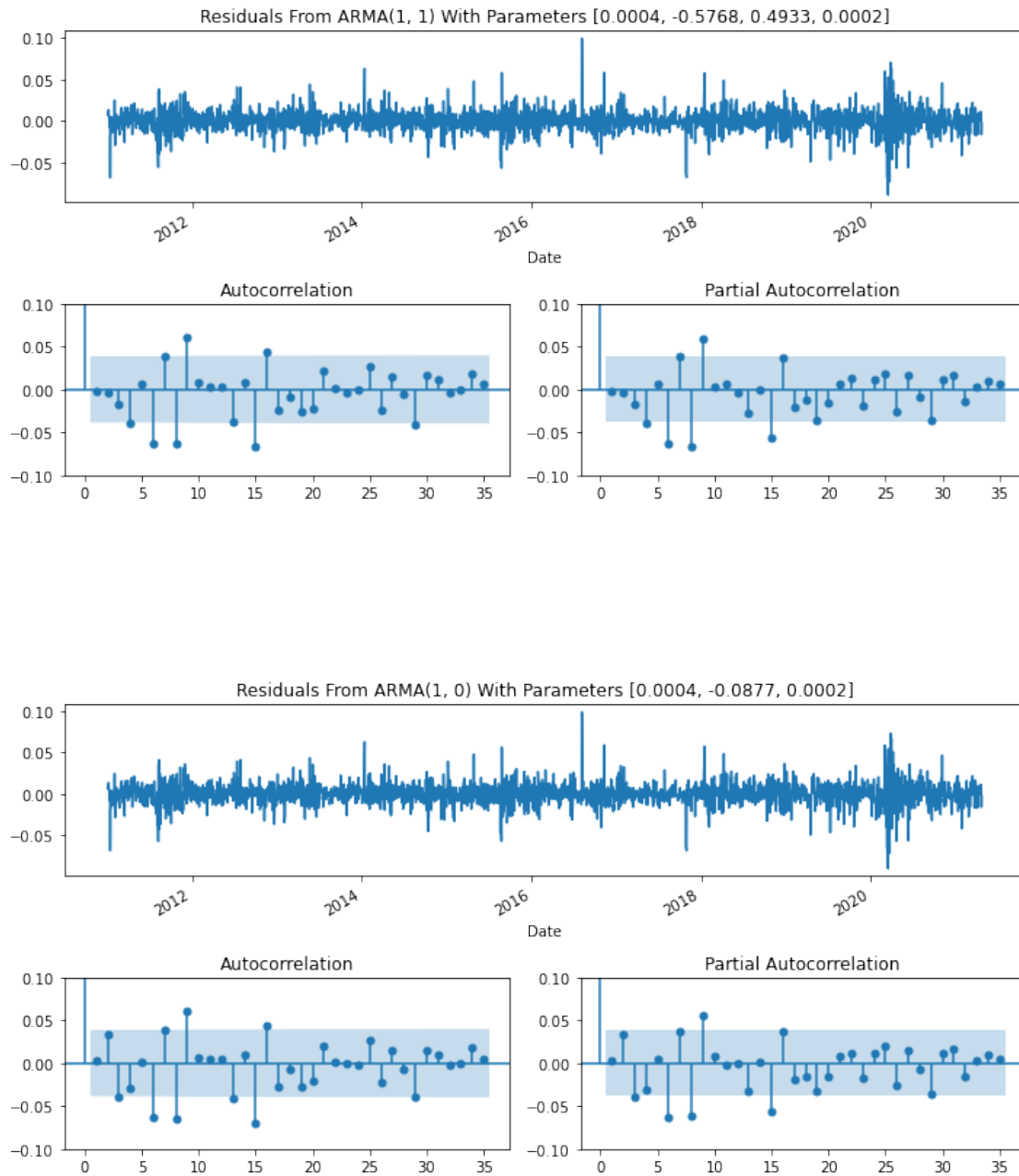
We then fit a few neighboring models of ARMA(4, 2). Based on the resulting ACF and PACF, and favoring low-order models, we will continue our analysis with ARMA(2, 1).

```
[5]: for p, q in [(4, 2), (4, 1), (3, 2), (3, 1), (3, 0), (2, 1), (1, 1), (1, 0)]:
    model = ARIMA(logret, order=(p, 0, q)).fit()
    tsplot(ts=model.resid, title=f"Residuals From ARMA{(p, q)} With Parameters",
    ↪{[round(param, 4) for param in model.params.values]}", acf_ylim=(-0.1, 0.1))
    print()
```









Again we fit an ARMA(2, 1) model to the data. Below are the estimated parameters.

```
[4]: from statsmodels.tsa.arima.model import ARIMA

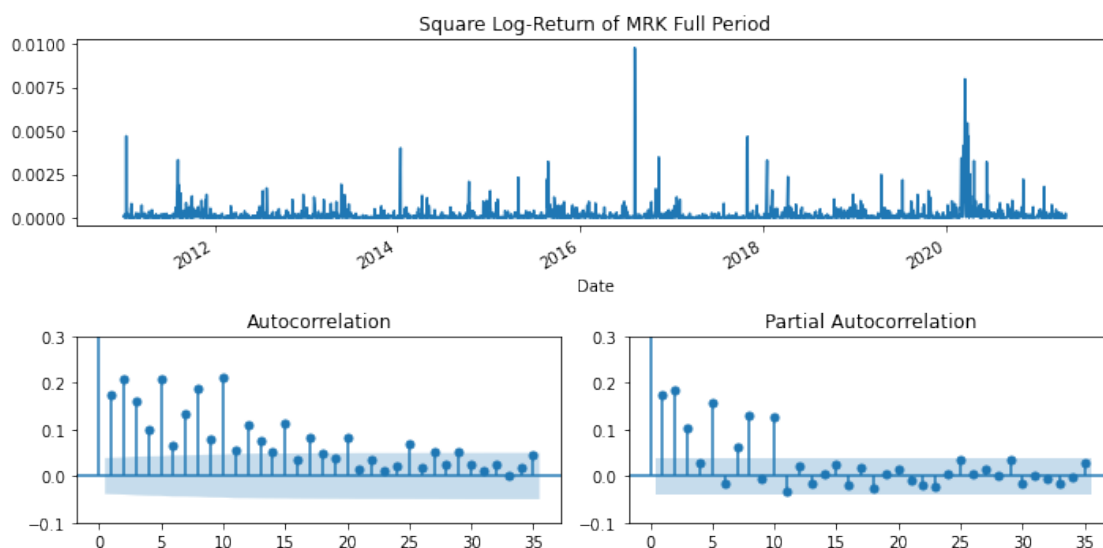
model = ARIMA(logret, order=(2, 0, 1)).fit()
model.params
```

```
[4]: const      0.000426
      ar.L1     -0.998822
      ar.L2     -0.088264
      ma.L1      0.910834
      sigma2    0.000167
      dtype: float64
```

(d)

From the below ACF plot, the residual time series of the fitted ARMA(2, 1) clearly has nonzero autocorrelations. This indicates that the residuals have the ARCH effect.

```
[10]: tsplot(model.resid**2, title='Square Log-Return of MRK Full Period',
            ↪acf_ylim=(-0.1, 0.3))
```



The Ljung-Box test results also confirm the existence of nonzero autocorrelation, starting lag 1.

```
[11]: acorr_ljungbox(model.resid**2, lags=10, return_df=True)
```

```
[11]:
```

	lb_stat	lb_pvalue
1	79.059515	6.026677e-19
2	190.797676	3.705195e-42
3	256.100817	3.135058e-55
4	281.747781	9.357271e-60
5	394.517629	4.506114e-83
6	405.459638	1.873476e-84
7	451.294789	2.340983e-93
8	541.970608	6.887799e-112

```

9    558.248528  1.896618e-114
10   675.128525  1.367979e-138

```

The Engle test result obtained by `statsmodels.stats.diagnostic.het_arch` also confirms the existence of the ARCH effect.

```

[5]: from statsmodels.stats.diagnostic import het_arch

het_arch(model.resid)

```

```

[5]: (366.5734209993779,
      1.1422821327788142e-60,
      15.102200243285889,
      4.964401683587653e-66)

```

(e)

We fit a GARCH(1, 1) model to the data. Below is a summary of the fitted model.

```

[11]: from arch.univariate import arch_model

am = arch_model(model.resid)
res = am.fit(update_freq=5)
res.summary()

```

```

Optimization terminated successfully    (Exit mode 0)
Current function value: -7763.966589888638
Iterations: 6
Function evaluations: 18
Gradient evaluations: 2

```

```

[11]: <class 'statsmodels.iolib.summary.Summary'>
      """

```

```

              Constant Mean - GARCH Model Results
=====
Dep. Variable:          None    R-squared:                0.000
Mean Model:             Constant Mean    Adj. R-squared:         0.000
Vol Model:              GARCH    Log-Likelihood:        7763.97
Distribution:           Normal    AIC:                  -15519.9
Method:                Maximum Likelihood    BIC:                  -15496.5
                               No. Observations:          2593
Date:                  Wed, Apr 28 2021    Df Residuals:          2592
Time:                  21:42:54    Df Model:              1
                               Mean Model
=====
              coef      std err          t      P>|t|      95.0% Conf. Int.
-----

```

```

mu          3.5155e-04  2.451e-04      1.434      0.151 [-1.288e-04,8.319e-04]
              Volatility Model
=====
              coef      std err          t      P>|t|      95.0% Conf. Int.
-----
omega        1.6729e-05  2.632e-12  6.356e+06      0.000 [1.673e-05,1.673e-05]
alpha[1]      0.1000  2.642e-02      3.785  1.534e-04  [4.822e-02, 0.152]
beta[1]       0.8000  2.576e-02     31.055  9.695e-212  [ 0.750, 0.850]
=====

Covariance estimator: robust
"""

```

(f)

Once again we fit the ARMA(2, 1)+GARCH(1, 1) model, but this time only to the data up to 4/1/2021. The resulting model is very similar to the ones above.

```

[3]: import datetime

dates = [date.date() for date in logret.index]
print('Index of 4/1/2021: ', dates.index(datetime.date(2021, 4, 1)))
print('Index of 4/23/2021: ', dates.index(datetime.date(2021, 4, 23)))

```

```

Index of 4/1/2021: 2577
Index of 4/23/2021: 2592

```

```

[4]: from statsmodels.tsa.arima.model import ARIMA

model = ARIMA(logret[:2578], order=(2, 0, 1)).fit()
model.params

```

```

[4]: const      0.000425
     ar.L1      -1.002473
     ar.L2      -0.088619
     ma.L1       0.914535
     sigma2     0.000168
     dtype: float64

```

```

[5]: from arch.univariate import arch_model

am = arch_model(model.resid)
res = am.fit(update_freq=5)
res.params

```

```

Optimization terminated successfully    (Exit mode 0)
Current function value: -7714.44208670915

```

```
Iterations: 5
Function evaluations: 56
Gradient evaluations: 4
```

```
[5]: mu          0.000352
     omega       0.000017
     alpha[1]    0.100000
     beta[1]     0.800000
     Name: params, dtype: float64
```

Our goal here is to estimate σ_t on 4/23/2021 given information up to 4/1/2021. Let t be 4/1/2021. Following the lecture, the volatility forecast on 4/5/2021 (next trading date of 4/1/2021) is a one-step ahead forecast

$$\hat{\sigma}_{t+1} = \sqrt{\omega + \alpha_1 a_t^2 + \beta_1 \sigma_t^2},$$

where a_t is the last residual of the ARMA(2, 1) model, and σ_t is the last conditional volatility of the fitted GARCH(1, 1) model. The forecast is 0.01151.

```
[9]: import numpy as np

     omega = res.params['omega']
     alpha = res.params['alpha[1]']
     beta = res.params['beta[1]']

     a = model.resid[-1]
     sigma_prev = res.conditional_volatility[-1]

     sigma = np.sqrt(omega + alpha*(a**2) + beta*(sigma_prev**2))
     sigma
```

```
[9]: 0.011510854344148646
```

Following the lecture, the k -step ahead forecast is

$$\hat{\sigma}_{t+k} = \sqrt{\omega + (\alpha_1 + \beta_1) \hat{\sigma}_{t+k-1}^2},$$

where $\hat{\sigma}_{t+k-1}$ is the $(k-1)$ -step ahead forecast. Using this formula, we can forecast the volatility from 4/6/2021 to 4/23/2021. The result is as below.

```
[21]: import pandas as pd
     from pandas import DataFrame

     forecast = []
     for i in range(14):
         sigma = np.sqrt(omega + (alpha + beta)*(sigma**2))
         forecast.append((dates[2579 + i], sigma))

     pd.set_option('display.precision', 16)
```

```
DataFrame(forecast, columns=['Date', 'Volatility Forecast']).set_index('Date')
```

```
[21]:
```

Date	Volatility Forecast
2021-04-06	0.0129557560084002
2021-04-07	0.0129557560183255
2021-04-08	0.0129557560272582
2021-04-09	0.0129557560352976
2021-04-12	0.0129557560425331
2021-04-13	0.0129557560490451
2021-04-14	0.0129557560549058
2021-04-15	0.0129557560601805
2021-04-16	0.0129557560649277
2021-04-19	0.0129557560692002
2021-04-20	0.0129557560730454
2021-04-21	0.0129557560765062
2021-04-22	0.0129557560796208
2021-04-23	0.0129557560824240